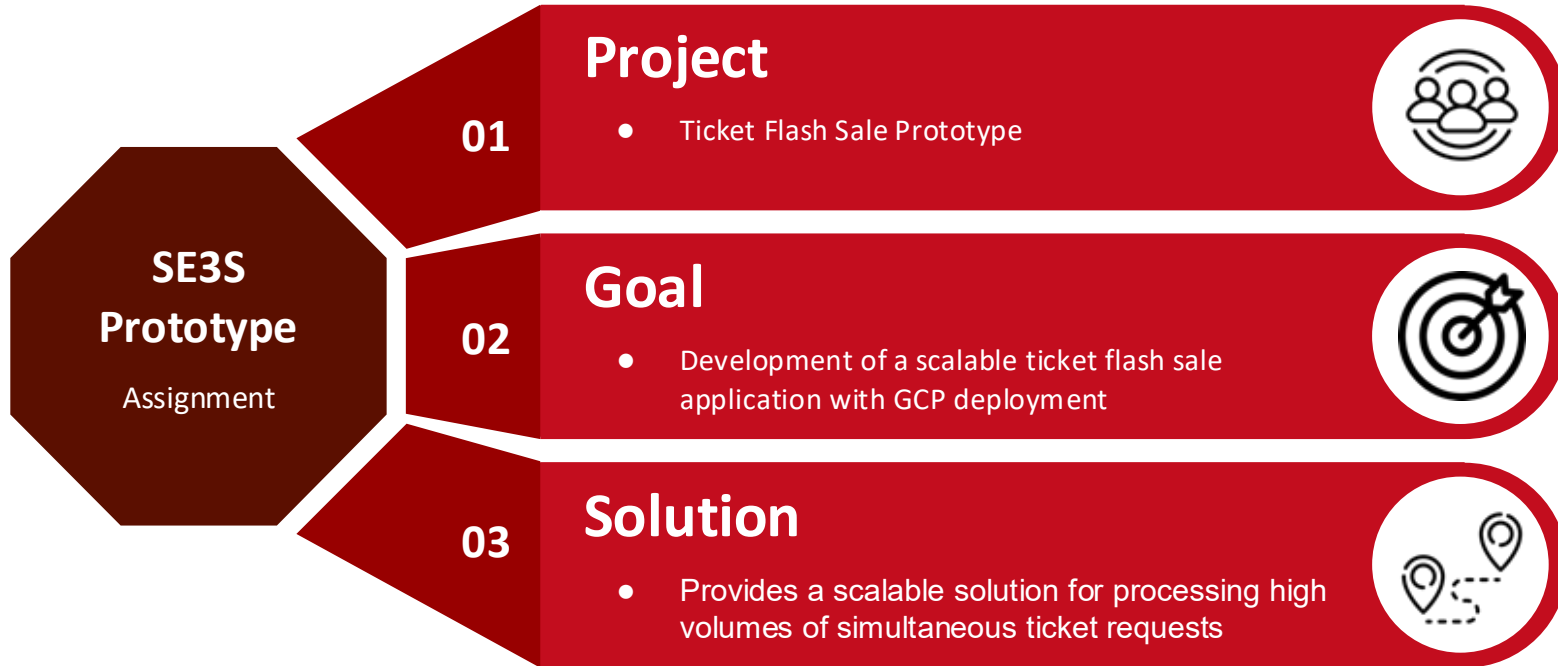


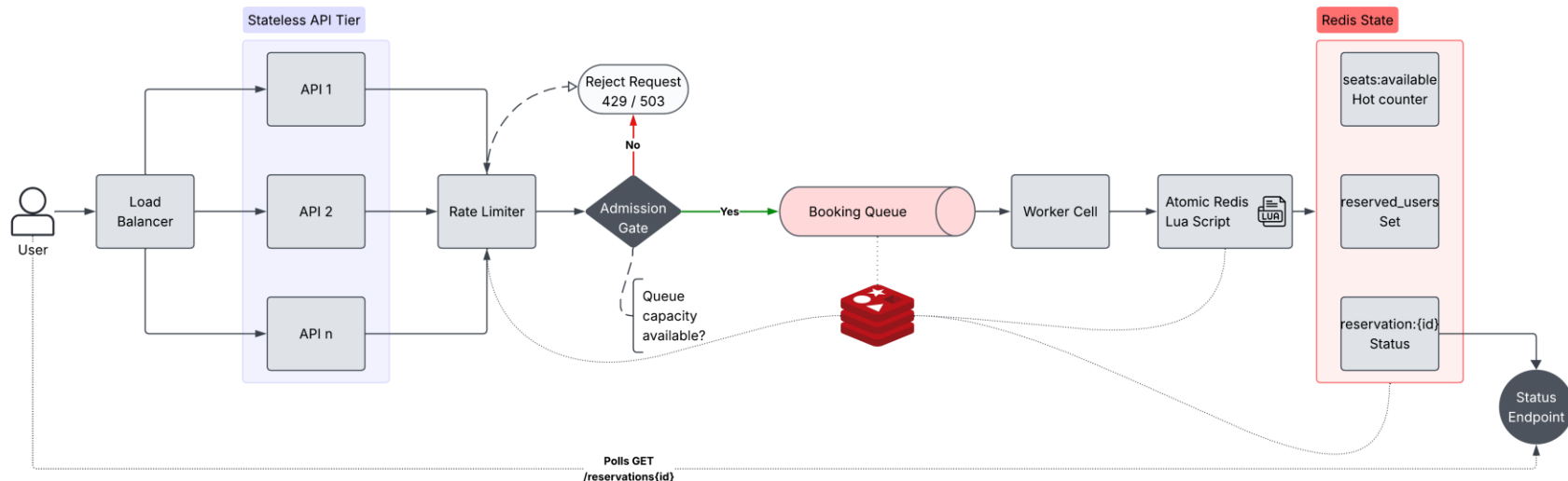


Scaling a Flash Sale Service

Felix Hauptmann, Ella Bredow, Efe Sonugür | Prototyping Presentation | July 12, 2026

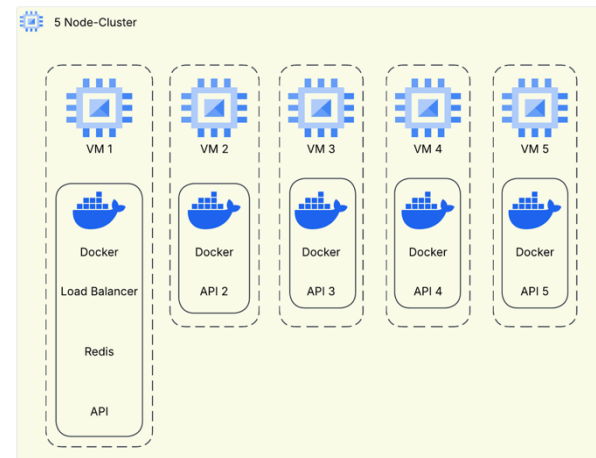
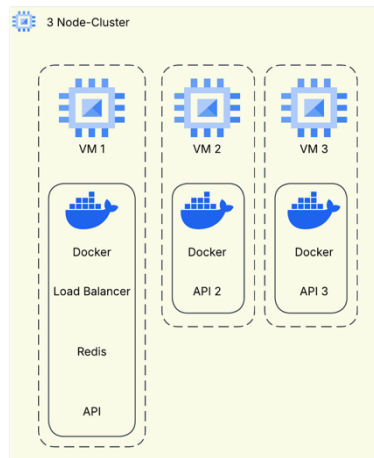
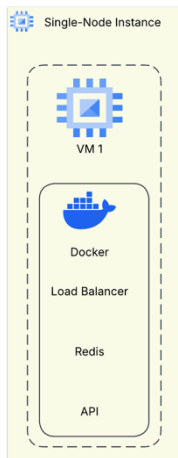


Application Architecture



Cluster Setup

- Scale FastAPI because Worker handle shared state
- Better scaling results by FastAPI
 - FastAPI handle short-lived request
 - Workers have longer jobs



Used Patterns

Constant Work

Workers:

- Processes a fixed number of queue items per interval

Bounded Queue

Admission Gate:

- Checks queue length & rejects if queue is full

Load Balancer

NGINX:

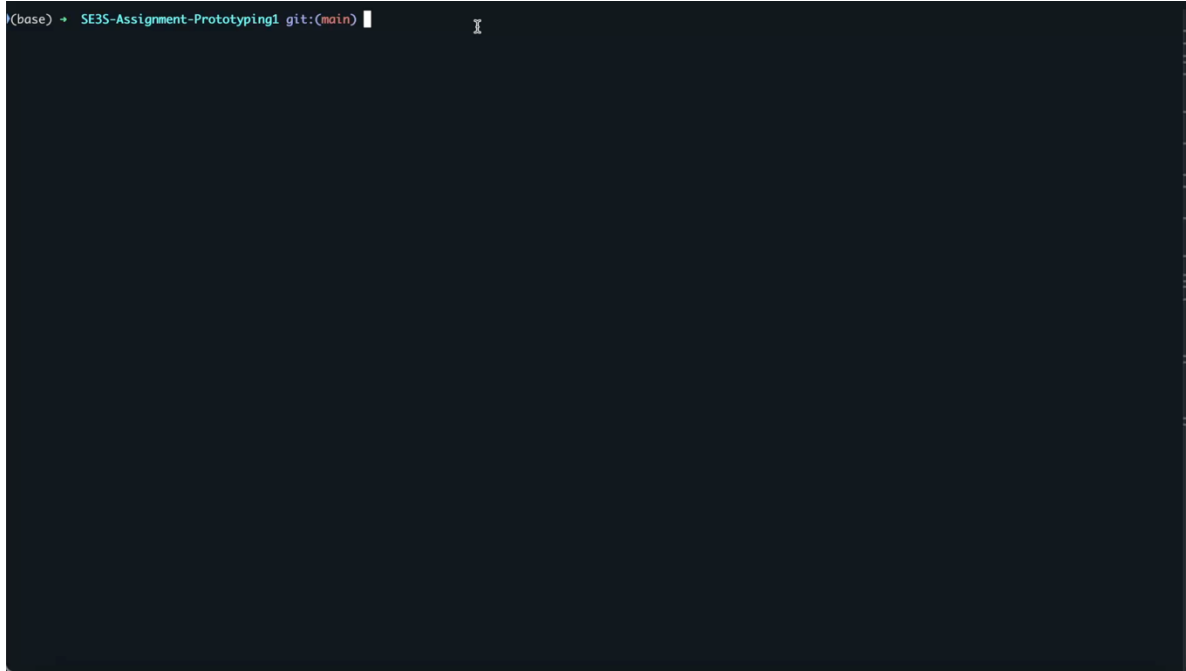
- Distributes requests using least connections

Token Bucket

Redis Rate Limiter:

- Limits repeated booking attempts per use

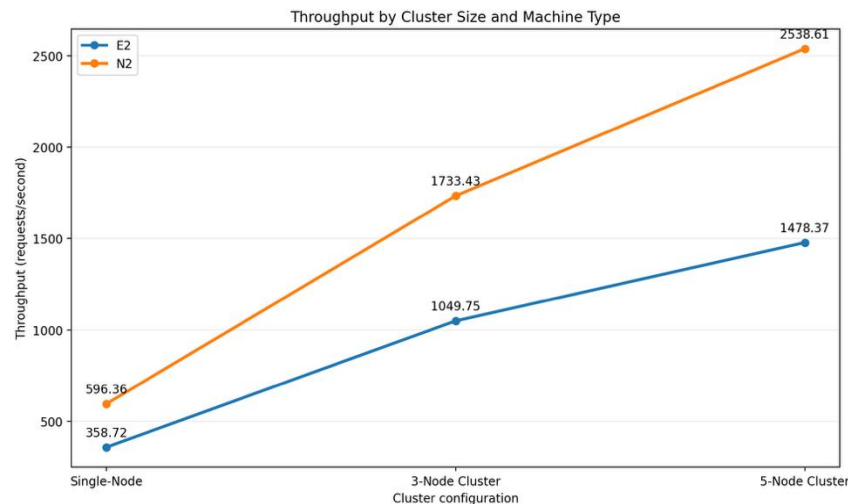
Demo



Scalability Results – Baseline Test

Cluster	Req/s – E2	Req/s – N2
Single-Node Instance	358.72	596.36
3 Node-Cluster	1049.75	1733.43
5 Node-Cluster	1478.37	2538.61

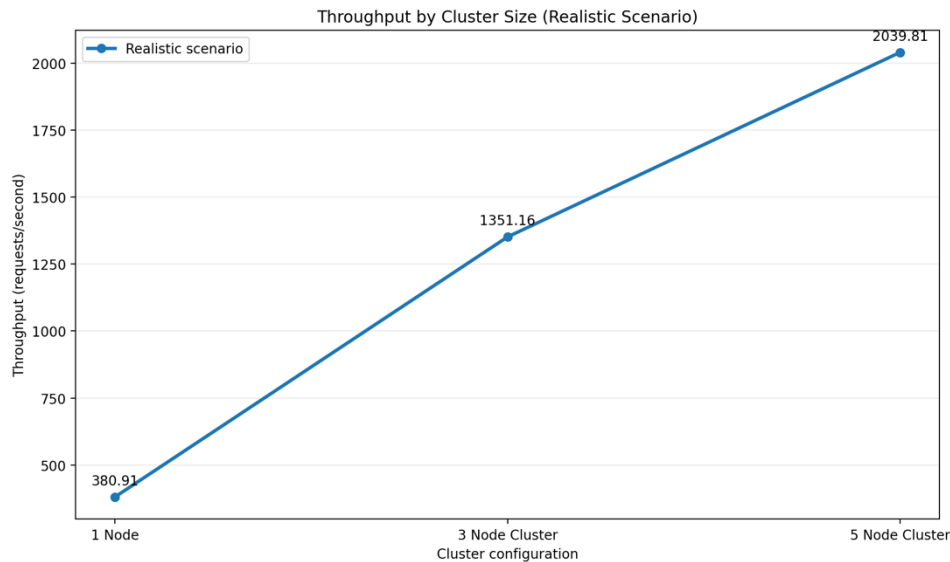
→ Almost Linear!



Scalability Results – Realistic Scenario Test

Cluster	Req/s – E2
Single-Node Instance	358.72
3 Node-Cluster	1049.75
5 Node-Cluster	1478.37

→ Confirms performance in a realistic scenario



Limitations & Possible Mitigation

Possible Bottlenecks:

- **Load Balancer**
 - Deploy multiple instances
- **Admission Gate**
 - Scaling requires more communication
- **Worker**
 - Possibility to also scale horizontally
- **Lua Script**
 - Mitigate by sharding

Optimize the Replica:

- Move high impact service to separate VM

Any questions?